REPORT · 2026年8月

造像类文物三维建模自动化系统 report+runner代码说明书

M6 报表 + M0 编排 / L3 调度 + L4 CLI 入口（含开发进度与下一步方向）

文保数字化研发组
2026年8月

目 录

一、概述 3

二、模块划分 4

三、依赖关系与数据流 5

3.1 依赖方向 5

3.2 批量调度数据流 5

四、模块详细设计 7

4.1 report/csv_export.py — CSV 导出 7

4.2 report/plot_gen.py — matplotlib 图表 7

4.3 report/__init__.py — generate 汇总入口 7

4.4 runner/task.py — 任务 / 阶段状态机 7

4.5 runner/batch_processor.py — M0 编排内核 8

4.6 runner/__init__.py — STAGE_REGISTRY + M1/M2 8

4.7 cli.py — h3d 命令入口 8

五、接口契约与错误码 10

六、测试与验证 11

七、当前开发进度与完成度 12

7.1 各层级完成情况 12

7.2 整体完成度 12

7.3 下一步开发方向 13

附录 A：run_manifest.json 示例 14

附录 B：failures.csv 与 metrics_summary.csv 示例 15

一、概述

本文档是《造像类文物三维建模自动化系统（路线 E）》**report（M6 报表，L2 业务层 / S4 阶段）+ runner（M0 编排 / L3 编排层 / S4 阶段）+ cli（L4 应用层）**的代码说明书。report 负责把各阶段 JSON 汇总为 CSV 指标与 matplotlib 图表（统一 utf-8-sig 编码 便于 Excel 直接打开）；runner 负责 M0 编排内核，按目录命名规约发现造像、注册阶段、驱动队列、失败隔离、断点续跑并落盘 run_manifest.json；cli 是 L4 命令入口，init / validate / batch / report 四子命令打通端到端。

设计要点：**失败隔离**单尊失败仅跳过该尊（不中断队列），**断点续跑**通过 run_manifest 判定已完成阶段直接跳过，**结果统一**每个阶段返回值统一为 StageResult{outputs, metrics, elapsed_s}，报表层据此一键出 CSV / 图表。

二、模块划分

report + runner + cli 共 8 个文件，职责与对外符号如下：

```
src/h3d/report/
+-- __init__.py     # 包导出 + generate() 汇总入口
+-- csv_export.py   # CSV 指标导出（含 failures.csv）
+-- plot_gen.py     # matplotlib 图表（柱状 / 折线）

src/h3d/runner/
+-- __init__.py     # STAGE_REGISTRY + register_stage + M1/M2 阶段 + run_batch
+-- task.py         # TaskStatus / StageResult / Task 状态机
+-- batch_processor.py  # discover_items + BatchProcessor 失败隔离 / 续跑 / manifest

src/h3d/cli.py      # h3d init / validate / batch / report 命令入口
```

模块文件	核心职责	对外关键符号
report/csv_export.py	CSV 写出（utf-8-sig 兼容 Excel）+ failures.csv + 阶段汇总	write_csv, export_failures_csv, export_manifest_csv
report/plot_gen.py	matplotlib 图表（无头 Agg + 中文字体自适应）	plot_bar, plot_stage_timings, plot_metric_series
report/__init__.py	报表汇总入口（配置驱动，CSV + PNG 原子写）	generate(manifest, failures, cfg)
runner/task.py	任务级 / 阶段级状态机（PENDING/RUNNING/DONE/FAIL/SKIPPED）	TaskStatus, StageResult, Task
runner/batch_processor.py	M0 编排内核：发现任务、循环调度、失败隔离、续跑、manifest	discover_items, BatchProcessor, BatchResult
runner/__init__.py	阶段注册表 + 预注册 M1/M2 + 批量处理入口	STAGE_REGISTRY, register_stage, run_batch
cli.py	h3d 命令入口（init/validate/batch/report），统一退出码	main, _cmd_init, _cmd_validate, _cmd_batch, _cmd_report

三、依赖关系与数据流

3.1 依赖方向

report + runner + cli 均依赖 core 底座与已实现的业务子包（io / preprocess / register），不反向，符合《骨架分析》5.2 依赖矩阵：

```
package-level deps (A -> B means A imports B):
  report     -->  core.exceptions (ExportError, H3DError)
  report     -->  core.metadata   (RunManifest)
  report     -->  core.config     (ReportConfig)
  report     -->  matplotlib      (惰性导入，运行时按需)

  runner     -->  core.exceptions (RecoverableError, H3DError)
  runner     -->  core.metadata   (RunManifest, sha256_text)
  runner     -->  core.config     (PipelineConfig)
  runner     -->  core.logging    (get_logger)
  runner     -->  h3d.io / h3d.preprocess / h3d.register  (M1/M2 阶段实装)

  cli        -->  core.config / core.logging / core.exceptions / core.metadata
  cli        -->  runner / report (业务调度分发)

  (core / io / preprocess / register)  -->  (NONE; 不反向依赖)
```

3.2 批量调度数据流

``run_batch(cfg)`` 端到端数据流（自顶向下）：



run_batch() 编排数据流：发现任务 → 加载/新建 manifest → 逐尊逐阶段调度 → 失败隔离 / 断点续跑 / manifest 记录 → 落盘 → 报表生成

四、模块详细设计

4.1 report/csv_export.py — CSV 导出

所有 CSV 统一使用 **utf-8-sig** 编码（带 BOM），保证 Excel 双击直接打开中文不乱码；缺数据列留空（《软著方案》2.6「单项缺失 -> 该列留空」）。failures.csv 字段严格对齐《软著方案》2.7.4（文物编号 / 阶段 / 异常类型 / 异常信息 / 时间）。

函数	职责
write_csv(rows, path, fieldnames)	通用 CSV 写出原语，支持 [(key, 中文表头)] 控制列序
export_failures_csv(failures, path)	失败清单写出（中文表头、固定字段顺序）
export_manifest_csv(manifest, path)	把 manifest.stages 拆为 (item_id, stage) + 状态/耗时/产物/指标

4.2 report/plot_gen.py — matplotlib 图表

固定使用 **Agg 无头后端**（服务器无显示器也能渲染），通过 fontManager 注册 Windows 中文字体（微软雅黑 → 黑体），关闭 axes.unicode_minus 避免负号豆腐块。

函数	职责
plot_bar(labels, values, path, ...)	通用柱状图原语（标题/坐标轴/网格）
plot_stage_timings(manifest, path)	各阶段平均耗时柱状图（按 stage 聚合）
plot_metric_series(manifest, metric, path)	单指标散点 / 折线（如 fitness、inlier_rmse）

4.3 report/__init__.py — generate 汇总入口

报表生成由 :func:`generate` 统一调度，按 ``ReportConfig.formats`` 过滤（csv / png），空 failures 不写 failures.csv，与软著方案 2.6「单件失败不影响报表」一致。

```
def generate(manifest, failures=None, report_cfg=None):
    report_cfg = report_cfg or ReportConfig()
    if not report_cfg.enabled: return []
    out_dir = report_cfg.output_dir
    os.makedirs(out_dir, exist_ok=True)
    outputs = []
    if 'csv' in report_cfg.formats:
        outputs.append(export_manifest_csv(manifest,
            os.path.join(out_dir, 'metrics_summary.csv')))
        if failures:
            outputs.append(export_failures_csv(failures,
                os.path.join(out_dir, 'failures.csv')))
    if 'png' in report_cfg.formats:
        outputs.append(plot_stage_timings(manifest,
            os.path.join(out_dir, 'stage_timings.png')))
    return outputs
```

4.4 runner/task.py — 任务 / 阶段状态机

:class:`TaskStatus` 是字符串枚举（值可直接序列化到 manifest），五状态：PENDING / RUNNING / DONE / FAIL / SKIPPED。:class:`Task` 以 ``results`` 字典按阶段记录 :class:`StageResult`（产出物 + 指标 + 耗时 + 错误）。任务级状态由各阶段状态推导：任一阶段失败则任务失败，全部完成（或跳过）则任

务完成。

方法	职责
begin_stage(stage)	标记阶段进入运行态，任务级 status=RUNNING
finish_stage(stage, outputs, metrics, elapsed_s)	阶段完成，登记产出与指标，任务级 status=DONE
fail_stage(stage, error)	阶段失败，联动任务级 status=FAIL（失败隔离据此跳过后续阶段）
mark_skipped(stage)	断点续跑：标记阶段跳过，不重复计算
is_done(stage) / outputs_of(stage) / metrics_of(stage)	查询接口，供 M2 阶段读 M1 产出物

4.5 runner/batch_processor.py — M0 编排内核

由 :func:`discover_items` + :class:`BatchProcessor` 两部分组成。discover_items 按《软著方案》2.7.1 命名规约扫描 ``raw_dir``：若存在子目录则每子目录 = 一尊造像，否则每个匹配文件 = 一尊。BatchProcessor.run 依次：load/init manifest → 遍历 tasks → 遍历 stages → 断点续跑判定 → 执行 / 异常处理 → manifest 记录 → 落盘。

```
def _run_task(self, task, manifest, failures):
    for stage_id, stage_fn in self.stages.items():
        key = f'{task.item_id}/{stage_id}'
        if self.cfg.runner.resume and manifest.is_done(key):
            task.mark_skipped(stage_id); continue
        task.begin_stage(stage_id)
        try:
            res = stage_fn(task, self.cfg)
            task.finish_stage(stage_id, res.outputs, res.metrics, ...)
            manifest.record(key, 'done', res.outputs, elapsed, **res.metrics)
        except RecoverableError as exc:
            self._isolate(task, stage_id, key, exc, failures, manifest); break
        except Exception as exc:
            if not self.cfg.runner.fail_isolation: raise
            self._isolate(task, stage_id, key, exc, failures, manifest); break
```

4.6 runner/__init__.py — STAGE_REGISTRY + M1/M2

:data:`STAGE_REGISTRY` 是有序字典（保持注册顺序 = 执行顺序），:func:`register_stage` 装饰器把业务阶段函数挂入。本模块预注册了 M1 预处理 / M2 配准两个主线阶段（与io / preprocess / register 三个业务子包通过 **惰性导入** 隔离，避免 import h3d.runner 时即拖入 open3d 重型库）。

阶段	输入	输出	异常码
M1 预处理	task.raw_paths (00_raw/<item>/*)	10_clean/<item>/<item>_sNN.ply	E100 空点云
M2 配准	task.outputs_of('M1') 或 raw_paths	20_registered/<item>/<item>_sNN.ply	E100/E200

4.7 cli.py — h3d 命令入口

L4 应用层薄壳，argparse 解析 + 调度分发，与 GUI 共用同一套 runner / report 内核（满足《设计书》6.4「一致性」要求）。四子命令覆盖：项目初始化、配置与数据校验、批量无人值守处理、仅重新生成报表。统一退出码：0 成功 / 1 配置或文件错误 / 130 Ctrl-C 中断。

命令	功能	对应软著方案 §2.9
h3d init -p <dir>	初始化项目目录结构（00_raw / 10_clean / ...）	init
h3d validate -c cfg.yaml	校验配置合法性 + 扫描数据完整性（不处理）	validate
h3d batch -c cfg.yaml [--dry-run]	批量处理全部造像（无人值守）	batch / run
h3d report -c cfg.yaml	仅重新生成报表（不重跑处理）	report

五、接口契约与错误码

所有阶段函数统一返回 :class:`StageResult`（outputs/metrics/elapsed_s），所有 manifest 阶段条目以 ``item_id/stage`` 为复合键，错误码严格对齐《设计书》13.3：

错误码	异常类	触发场景	处理策略
E001	ConfigError	配置文件缺失 / 字段非法	main() 捕获 → 退出码 1
E100	InputDataError (RecoverableError)	空点云 / 输入文件缺失	_isolate: 记录 failures + 跳过该尊 + 继续队列
E200	RegistrationError (RecoverableError)	配准失败（重叠不足）	_isolate: 记录 + 跳过 + 继续
E601	ExportError	CSV / 图表写出失败（目录不可写）	main() 捕获 → 退出码 1

六、测试与验证

report 21 项、runner 26 项、cli 6 项、合计 53 项新测试，连同既有 core / io / preprocess / register 共 **129 项** 全部通过。覆盖：CSV 写出 / 图表渲染 / 状态机转换 / 任务发现 / 失败隔离 / 断点续跑 / manifest 持久化 / 端到端 CLI。

```
$ python -m pytest -q
..................................................................... [ 55%]
.....................................................                [100%]
129 passed in 5.98s

# 端到端冒烟（demo 项目：2 尊造像 × 2 站）
$ python -m h3d.cli validate -c demo.yaml   # 发现 2 尊 / 4 站
$ python -m h3d.cli batch    -c demo.yaml   # 全部 done，生成 manifest + 报表
$ python -m h3d.cli batch    -c demo.yaml   # 第二次：断点续跑，全部 SKIPPED
```

七、当前开发进度与完成度

本节梳理整条管线的开发进度、整体完成度与下一步开发方向，便于统筹安排。

7.1 各层级完成情况

层级 / 子包	对应设计书章节	完成状态	测试覆盖
L0 底座 core（exceptions / config / logging / metadata）	§12-13	已交付	29 项
L1 适配层 io（pointcloud_io / mesh_io）	§14	已交付	20 项
L2 业务层 preprocess（M1 接入 / 降采样 / 去噪 / 法线）	§32	已交付	16 项
L2 业务层 register（M2 FPFH+RANSAC+ICP）	§15 / §33	已交付	11 项
L2 业务层 report（M6 CSV + 图表）	§18 / 软著§2.6	**本次交付**	21 项
L3 编排层 runner（任务 / 调度 / 失败隔离 / 续跑）	§31 / 软著§2.7	**本次交付**	26 项
L4 应用层 cli（init / validate / batch / report）	§31 / 软著§2.9	**本次交付**	6 项
L2 业务层 mesh（M3 网格重建 / Poisson）	§16 / §34	未开始	—
L2 业务层 texture（M4 纹理映射）	§17	未开始	—
L2 业务层 quality（M5 质检质量门）	§35	未开始	—
L2 业务层 change（X1 M3C2 多时相）	§36	未开始	—
L2 业务层 colour（X2 色卡校准 + CIEDE2000）	§37	未开始	—

7.2 整体完成度

主干管线（core + io + preprocess + register + report + runner + cli）已贯通，端到端可跑通：M1 → M2 → manifest → 报表。 53 项新测试 + 既有 76 项共 129 项 pytest 全绿，CLI 冒烟两尊造像验证断点续跑与失败隔离均符合设计。

维度	完成度	说明
主线模块（M0-M2 + M6）	约 70%	M0 编排 + M1 接入 + M2 配准 + M6 报表已交付；M3 重建 / M4 纹理 / M5 质检待补
可选模块（X1 / X2）	0%	多时相变化检测 / 色彩量化按需启用
测试覆盖（既有 + 新增）	100% 通过率	core 29 + io 20 + preprocess 16 + register 11 + report 21 + runner 26 + cli 6 = 129 项

维度	完成度	说明
许可证合规	100% MIT/BSD	Open3D MIT / NumPy BSD-3 / laspy BSD-2 / PyYAML MIT / matplotlib PSF / 全部在白名单内

7.3 下一步开发方向

建议按以下顺序推进（从最重要到最次要）：

优先级	模块	目标	关键点
P0	M3 网格重建（mesh）	Poisson / Ball Pivoting 从点云重建网格	复用 STAGE_REGISTRY：@register_stage('M3')，与 M1/M2 同样无侵入接入
P0	M5 质检（quality）	中误差 / 覆盖率 / 孔洞率，触发 QualityGateError(E500)	质量门失败属 RecoverableError，自动进入 failures.csv
P1	M4 纹理（texture）	纹理映射 + 多分辨率 LOD 输出 OBJ/GLB/PLY	LOD_levels 与 export_formats 配置驱动
P1	run_manifest 增强	实现 .done 标记文件（《软著方案》2.7.3 幂等性）	manifest 已是续跑真相源，.done 标记可作冗余校验或离线审计
P2	并发调度	按 cfg.runner.workers 用 ProcessPoolExecutor 并行处理独立 item	现为串行实现保证可复现；并发需处理 open3d 在子进程中的初始化与 manifest 写冲突
P2	HTML 报表	report.formats 增加 html：汇总 + 嵌入图表的交互式报告	可在 jinja2 模板 + matplotlib SVG 嵌入实现
P3	X1 change（M3C2）	py4dgeo 距离场 + LoD 不确定性 + 显著性	py4dgeo 已在 venv_lab 中可装；变更检测为多时相场景
P3	X2 colour（CIEDE2000）	色卡校准 + 色彩量化 + ΔE 统计	colour-science 已在 venv_lab 中可装

建议路径：先 M3（mesh）→ M5（quality）→ M4（texture），把「点云 → 模型」的核心闭环全部跑通；再视场景需要启用 X1（多时相监测）或 X2（色彩量化）。并发调度作为单独的性能优化项，在主线稳定后实施。

附录 A：run_manifest.json 示例

一次成功批量处理（2 尊造像 × M1 + M2 共 4 个阶段条目）生成的 run_manifest.json 节选：


```
{
  "run_id": "20260825-run-5srd",
  "created_at": "2026-08-25T15:08:03",
  "config_sha256": "9f86d081884c7d659a2feaa0c55ad015a3bf4f1b2b0b822cd15d6c15b0f00a08",
  "stages": {
    "rock001/M1": {"status": "done", "outputs": ["...rock001_s01.ply", "...rock001_s02.ply"],
                   "elapsed_s": 1.085, "n_scans": 2, "points_in": 3000, "points_out": 2928},
    "rock001/M2": {"status": "done", "outputs": ["...rock001_s02.ply"],
                   "elapsed_s": 0.041, "n_scans": 2, "registered": true,
                   "fitness_s02": 1.0, "rmse_s02": 0.0, "fitness": 1.0, "inlier_rmse": 0.0},
    "rock002/M1": {...},
    "rock002/M2": {...}
  },
  "software": {"open3d": "0.19.0", "numpy": "1.26.x", "h3d": "0.1.0"}
}
```

附录 B：failures.csv 与 metrics_summary.csv 示例

failures.csv（失败清单，对齐《软著方案》2.7.4）：

```
文物编号,阶段,异常类型,异常信息,时间
rock003,M1,InputDataError,点云为空 0 点,2026-08-25T15:08:30
rock005,M2,RegistrationError,重叠不足 fitness=0.08,2026-08-25T15:09:12
```

metrics_summary.csv（阶段汇总，Excel 直接透视）：

```
文物编号,阶段,状态,耗时(s),产物路径,关键指标
rock001,M1,done,1.085,10_clean/rock001/...,n_scans=2; points_in=3000; points_out=2928
rock001,M2,done,0.041,20_registered/rock001/...,fitness=1.0; inlier_rmse=0.0
rock002,M1,done,0.084,10_clean/rock002/...,n_scans=2; points_in=3000; points_out=2928
rock002,M2,done,0.040,20_registered/rock002/...,fitness=1.0; inlier_rmse=0.0
```